

CRUD con Spring Boot + Maven y sus Pruebas

Guía paso a paso — Curso Estándares y Métricas de Calidad

UNIMINUTO — Especialización en Desarrollo de Software

Tabla de contenidos

- 1. [Requisitos previos](#)
- 2. [Estructura del proyecto](#)
- 3. [Configuración Maven \(pom.xml\)](#)
- 4. [Capa de Modelo](#)
- 5. [Capa de Repositorio](#)
- 6. [Capa de Servicio](#)
- 7. [Capa de Controlador \(REST\)](#)
- 8. [Configuración de la aplicación](#)
- 9. [Clase principal](#)
- 10. [Pruebas unitarias \(Mockito\)](#)
- 11. [Pruebas de integración \(MockMvc\)](#)
- 12. [Pruebas de aceptación \(Cucumber BDD\)](#)
- 13. [Cómo ejecutar todo](#)
- 14. [Tabla comparativa de pruebas](#)

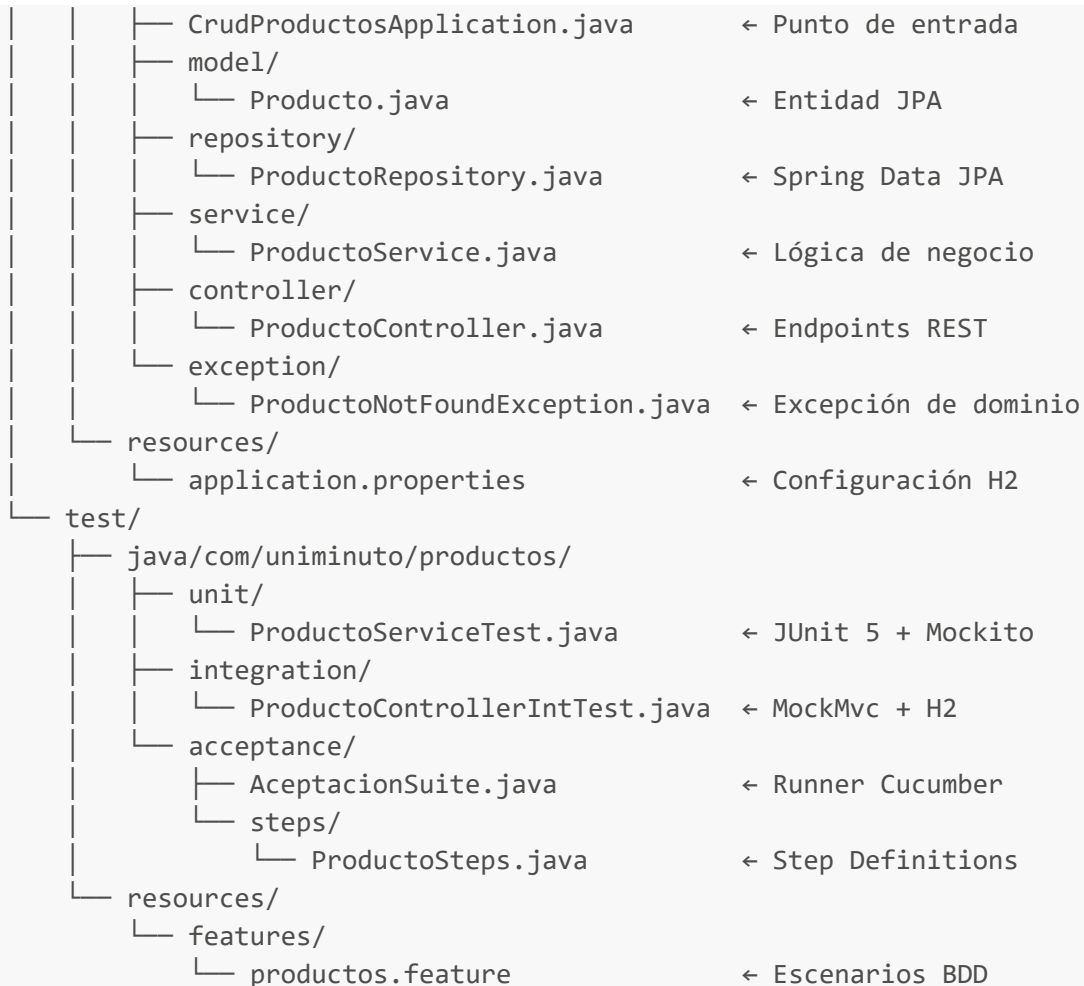
1. Requisitos previos

Herramienta	Versión mínima	Verificar con
Java JDK	17	<code>java -version</code>
Maven	3.8+	<code>mvn -version</code>
IntelliJ IDEA	2024+	—
(Opcional) Postman	cualquiera	Para probar endpoints

La base de datos usada es **H2 en memoria** — no requiere instalación de MySQL ni PostgreSQL para correr el proyecto.

2. Estructura del proyecto

```
crud-productos/  
├─ pom.xml  
└─ src/  
    ├─ main/  
    │   └─ java/com/uniminuto/productos/
```



3. Configuración Maven (pom.xml)

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <!-- Spring Boot como padre: gestiona versiones de todas las dependencias -->
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.5</version>
    <relativePath/>
  </parent>

  <groupId>com.uniminuto</groupId>
  <artifactId>crud-productos</artifactId>
  <version>1.0.0</version>
  <name>CRUD Productos - Spring Boot</name>

  <properties>

```

```
<java.version>17</java.version>
<cucumber.version>7.16.1</cucumber.version>
</properties>

<dependencies>

  <!-- — PRODUCCIÓN ————— -->

  <!-- Web: incluye Spring MVC + Tomcat embebido -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>

  <!-- JPA: ORM con Hibernate para persistencia -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- Validation: anotaciones @NotNull, @Size, @Email, etc. -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>

  <!-- H2: base de datos en memoria (también útil en pruebas) -->
  <dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>runtime</scope>
  </dependency>

  <!-- Lombok: elimina boilerplate (getters, setters, constructores) -->
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>

  <!-- — PRUEBAS ————— -->

  <!-- Spring Boot Test: incluye JUnit 5, Mockito, MockMvc, AssertJ -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
  </dependency>

  <!-- Cucumber: motor BDD para pruebas de aceptación -->
  <dependency>
    <groupId>io.cucumber</groupId>
    <artifactId>cucumber-java</artifactId>
    <version>${cucumber.version}</version>
```

```

        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>io.cucumber</groupId>
        <artifactId>cucumber-spring</artifactId>
        <version>${cucumber.version}</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>io.cucumber</groupId>
        <artifactId>cucumber-junit-platform-engine</artifactId>
        <version>${cucumber.version}</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.junit.platform</groupId>
        <artifactId>junit-platform-suite</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <!-- Plugin de Spring Boot: genera JAR ejecutable -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <excludes>
                    <exclude>
                        <groupId>org.projectlombok</groupId>
                        <artifactId>lombok</artifactId>
                    </exclude>
                </excludes>
            </configuration>
        </plugin>
    </plugins>
</build>

</project>

```

¿Por qué `spring-boot-starter-parent`? Centraliza la gestión de versiones compatibles entre sí. Sin él, habría que declarar manualmente cada versión y resolver conflictos de dependencias.

4. Capa de Modelo

Archivo: `src/main/java/com/uniminuto/productos/model/Producto.java`

```
package com.uniminuto.productos.model;

import jakarta.persistence.*;
import jakarta.validation.constraints.*;
import lombok.*;
import java.math.BigDecimal;

/**
 * ENTIDAD JPA: Producto
 *
 * @Entity → JPA creará la tabla "productos" automáticamente
 * @Table → nombre explícito de la tabla en BD
 * @Id → clave primaria
 *
 * Las anotaciones de validación (@NotBlank, @Positive) se activan
 * cuando el controlador recibe datos del cliente. Protegen la BD
 * sin requerir lógica extra en el servicio.
 */
@Entity
@Table(name = "productos")
@Data // Lombok: getters + setters + equals + hashCode + toString
@Builder
@NoArgsConstructor
@AllArgsConstructor
public class Producto {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "El nombre es obligatorio")
    @Size(min = 2, max = 100, message = "El nombre debe tener entre 2 y 100 caracteres")
    @Column(nullable = false, length = 100)
    private String nombre;

    @Size(max = 500)
    private String descripcion;

    @NotNull(message = "El precio es obligatorio")
    @Positive(message = "El precio debe ser mayor a cero")
    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal precio;

    @Min(value = 0, message = "El stock no puede ser negativo")
    @Column(nullable = false)
    private Integer stock;

    @Column(nullable = false)
    private Boolean activo = true;
}
```

5. Capa de Repositorio

Archivo: `src/main/java/com/uniminuto/productos/repository/ProductoRepository.java`

```
package com.uniminuto.productos.repository;

import com.uniminuto.productos.model.Producto;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;

import java.util.List;
import java.util.Optional;

/**
 * REPOSITARIO: Acceso a datos con Spring Data JPA
 *
 * Al extender JpaRepository<Producto, Long> obtenemos GRATIS:
 * save(), findById(), findAll(), deleteById(), count(), existsById()...
 *
 * Solo declaramos los métodos adicionales que necesitamos.
 * Spring genera el SQL automáticamente a partir del nombre del método.
 */
@Repository
public interface ProductoRepository extends JpaRepository<Producto, Long> {

    // Spring traduce esto a: SELECT * FROM productos WHERE activo = true
    List<Producto> findByActivoTrue();

    // SELECT * FROM productos WHERE nombre ILIKE '%texto%' AND activo = true
    List<Producto> findByNombreContainingIgnoreCaseAndActivoTrue(String nombre);

    // Verificar duplicados antes de insertar
    boolean existsByNombreIgnoreCase(String nombre);

    // Consulta JPQL personalizada (útil cuando el nombre del método sería muy largo)
    @Query("SELECT p FROM Producto p WHERE p.precio BETWEEN :min AND :max AND p.activo = true")
    List<Producto> findByRangoDePrecio(
        @org.springframework.data.repository.query.Param("min")
        java.math.BigDecimal min,
        @org.springframework.data.repository.query.Param("max")
        java.math.BigDecimal max
    );
}
```

Nota didáctica: `JpaRepository` es un ejemplo del patrón **Repository** de Domain-Driven Design (DDD). Desacopla la lógica de negocio del mecanismo de persistencia.

6. Capa de Servicio

Archivo: `src/main/java/com/uniminuto/productos/service/ProductoService.java`

```
package com.uniminuto.productos.service;

import com.uniminuto.productos.exception.ProductoNotFoundException;
import com.uniminuto.productos.model.Producto;
import com.uniminuto.productos.repository.ProductoRepository;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

/**
 * SERVICIO: Lógica de negocio de Productos
 *
 * @Service → Spring lo registra como bean y lo gestiona en el contexto
 * @Transactional → cada método corre dentro de una transacción BD.
 * Si lanza excepción → rollback automático.
 *
 * REGLA: El servicio no sabe nada de HTTP (no conoce Request ni Response).
 * Esa es responsabilidad del Controlador.
 *
 * DISEÑO PARA TESTEABILIDAD:
 * @RequiredArgsConstructor de Lombok genera el constructor con el repositorio,
 * lo que habilita inyección de dependencias → en tests se puede inyectar un mock.
 */
@Service
@Transactional
@RequiredArgsConstructor
public class ProductoService {

    private final ProductoRepository repositorio;

    // ----- CREATE -----

    /**
     * Registra un nuevo producto.
     * Regla de negocio: no pueden existir dos productos con el mismo nombre.
     */
    public Producto crear(Producto producto) {
        if (repositorio.existsByNombreIgnoreCase(producto.getNombre())) {
            throw new IllegalArgumentException(
                "Ya existe un producto con el nombre: " + producto.getNombre()
            );
        }
        producto.setActivo(true);
        return repositorio.save(producto);
    }
}
```

```
// ----- READ -----

/**
 * Lista solo productos activos (no eliminados lógicamente).
 * readOnly=true → Hibernate optimiza: no hace flush ni seguimiento de
cambios.
 */
@Transactional(readOnly = true)
public List<Producto> listarActivos() {
    return repositorio.findByActivoTrue();
}

@Transactional(readOnly = true)
public Producto buscarPorId(Long id) {
    return repositorio.findById(id)
        .orElseThrow(() -> new ProductoNotFoundException(id));
}

@Transactional(readOnly = true)
public List<Producto> buscarPorNombre(String nombre) {
    return repositorio.findByNombreContainingIgnoreCaseAndActivoTrue(nombre);
}

// ----- UPDATE -----

/**
 * Actualiza datos del producto.
 * Regla: si cambia el nombre, verificar que no colisione con otro.
 */
public Producto actualizar(Long id, Producto datosNuevos) {
    Producto existente = buscarPorId(id);

    boolean cambiaNombre = !existente.getNombre()
.equalsIgnoreCase(datosNuevos.getNombre());
    if (cambiaNombre &&
repositorio.existsByNombreIgnoreCase(datosNuevos.getNombre())) {
        throw new IllegalArgumentException(
            "El nombre '" + datosNuevos.getNombre() + "' ya está en uso"
        );
    }

    existente.setNombre(datosNuevos.getNombre());
    existente.setDescripcion(datosNuevos.getDescripcion());
    existente.setPrecio(datosNuevos.getPrecio());
    existente.setStock(datosNuevos.getStock());

    return repositorio.save(existente);
}

// ----- DELETE -----

/**
 * Eliminación LÓGICA (soft delete): marca activo=false.
```



```

    * El registro permanece en BD para auditoría e historial.
    * Ventaja: se puede recuperar. Desventaja: requiere filtrar en consultas.
    */
    public void desactivar(Long id) {
        Producto producto = buscarPorId(id);
        producto.setActivo(false);
        repositorio.save(producto);
    }

    /** Eliminación física. Usar solo cuando se requiera explícitamente. */
    public void eliminar(Long id) {
        buscarPorId(id); // lanza excepción si no existe
        repositorio.deleteById(id);
    }
}

```

7. Capa de Controlador (REST)

Archivo: `src/main/java/com/uniminuto/productos/controller/ProductoController.java`

```

package com.uniminuto.productos.controller;

import com.uniminuto.productos.exception.ProductoNotFoundException;
import com.uniminuto.productos.model.Producto;
import com.uniminuto.productos.service.ProductoService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;
import java.util.Map;

/**
 * CONTROLADOR REST: Endpoints HTTP para gestión de productos
 *
 * @RestController → combina @Controller + @ResponseBody
 *                  (serializa automáticamente objetos Java a JSON)
 * @RequestMapping → prefijo de todas las rutas: /api/productos
 *
 * VERBOS HTTP usados:
 * GET      → leer (sin efectos secundarios)
 * POST     → crear un nuevo recurso
 * PUT      → reemplazar un recurso completo
 * DELETE   → eliminar un recurso
 *
 * CÓDIGOS DE RESPUESTA HTTP:
 * 200 OK      → operación exitosa con cuerpo
 * 201 Created → recurso creado exitosamente

```

```

* 204 No Content → operación exitosa sin cuerpo (DELETE)
* 400 Bad Request → datos inválidos enviados por el cliente
* 404 Not Found → recurso no encontrado
* 409 Conflict → conflicto de negocio (ej: nombre duplicado)
*/
@RestController
@RequestMapping("/api/productos")
@RequiredArgsConstructor
public class ProductoController {

    private final ProductoService servicio;

    // — POST /api/productos —————
    // @Valid activa las validaciones del modelo (@NotBlank, @Positive, etc.)
    @PostMapping
    public ResponseEntity<Producto> crear(@Valid @RequestBody Producto producto) {
        Producto creado = servicio.crear(producto);
        return ResponseEntity.status(HttpStatus.CREATED).body(creado);
    }

    // — GET /api/productos —————
    @GetMapping
    public ResponseEntity<List<Producto>> listar() {
        return ResponseEntity.ok(servicio.listarActivos());
    }

    // — GET /api/productos/{id} —————
    @GetMapping("/{id}")
    public ResponseEntity<Producto> buscarPorId(@PathVariable Long id) {
        return ResponseEntity.ok(servicio.buscarPorId(id));
    }

    // — GET /api/productos/buscar?nombre=texto —————
    @GetMapping("/buscar")
    public ResponseEntity<List<Producto>> buscarPorNombre(
        @RequestParam String nombre) {
        return ResponseEntity.ok(servicio.buscarPorNombre(nombre));
    }

    // — PUT /api/productos/{id} —————
    @PutMapping("/{id}")
    public ResponseEntity<Producto> actualizar(
        @PathVariable Long id,
        @Valid @RequestBody Producto datos) {
        return ResponseEntity.ok(servicio.actualizar(id, datos));
    }

    // — DELETE /api/productos/{id} —————
    // Soft delete: desactiva el producto sin eliminar el registro
    @DeleteMapping("/{id}")
    public ResponseEntity<Void> eliminar(@PathVariable Long id) {
        servicio.desactivar(id);
        return ResponseEntity.noContent().build(); // 204
    }
}

```

```
// — MANEJO DE EXCEPCIONES —————

@ExceptionHandler(ProductoNotFoundException.class)
public ResponseEntity<Map<String, String>> handleNotFound(
    ProductoNotFoundException ex) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(Map.of("error", ex.getMessage()));
}

@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<Map<String, String>> handleConflict(
    IllegalArgumentException ex) {
    return ResponseEntity.status(HttpStatus.CONFLICT)
        .body(Map.of("error", ex.getMessage()));
}
}
```

8. Configuración de la aplicación

Archivo: `src/main/resources/application.properties`

```
# — Base de datos H2 en memoria —————
spring.datasource.url=jdbc:h2:mem:productosdb;DB_CLOSE_DELAY=-1
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

# — JPA / Hibernate —————
# create-drop: crea tablas al iniciar, las elimina al cerrar
# Opciones: validate | update | create | create-drop
spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

# — Consola H2 (útil en desarrollo para inspeccionar la BD) —————
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console

# — Nombre de la aplicación —————
spring.application.name=crud-productos
```

Acceder a la consola H2 en: <http://localhost:8080/h2-console> URL JDBC:
`jdbc:h2:mem:productosdb`

9. Clase principal

Archivo: `src/main/java/com/uniminuto/productos/CrudProductosApplication.java`

```
package com.uniminuto.productos;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * @SpringBootApplication combina:
 *   @Configuration      → esta clase puede definir beans
 *   @EnableAutoConfiguration → Spring configura automáticamente lo que detecta
 *   @ComponentScan      → escanea los paquetes hijos buscando @Service,
 *                         @Controller, etc.
 */
@SpringBootApplication
public class CrudProductosApplication {

    public static void main(String[] args) {
        SpringApplication.run(CrudProductosApplication.class, args);
    }
}
```

Excepción de dominio:

src/main/java/com/uniminuto/productos/exception/ProductoNotFoundException.java

```
package com.uniminuto.productos.exception;

public class ProductoNotFoundException extends RuntimeException {

    public ProductoNotFoundException(Long id) {
        super("Producto no encontrado con id: " + id);
    }
}
```

10. Pruebas unitarias (Mockito)

Archivo: src/test/java/com/uniminuto/productos/unit/ProductoServiceTest.java

```
package com.uniminuto.productos.unit;

import com.uniminuto.productos.exception.ProductoNotFoundException;
import com.uniminuto.productos.model.Producto;
import com.uniminuto.productos.repository.ProductoRepository;
import com.uniminuto.productos.service.ProductoService;
import org.junit.jupiter.api.*;
import org.junit.jupiter.api.extension.ExtendWith;
import org.mockito.*;
import org.mockito.junit.jupiter.MockitoExtension;
```

```

import java.math.BigDecimal;
import java.util.Optional;

import static org.assertj.core.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

/**
 * PRUEBAS UNITARIAS del ProductoService
 *
 *  Prueban: lógica de negocio pura
 *  No usan: Spring context, base de datos, red
 *  Usan mock del repositorio para aislar el servicio
 *
 * Patrón AAA: Arrange → Act → Assert
 * Ejecución: mvn test -Dtest=ProductoServiceTest
 */
@ExtendWith(MockitoExtension.class)
@DisplayName("ProductoService – Pruebas Unitarias")
class ProductoServiceTest {

    @Mock
    private ProductoRepository repositorio;

    @InjectMocks
    private ProductoService servicio;

    private Producto productoValido;

    @BeforeEach
    void setUp() {
        productoValido = Producto.builder()
            .nombre("Laptop Dell XPS")
            .descripcion("Laptop profesional 16GB RAM")
            .precio(new BigDecimal("2500000.00"))
            .stock(10)
            .build();
    }

    // — CREAR —

    @Test
    @DisplayName("✓ crear: guarda producto cuando el nombre no existe")
    void crear_nombreNuevo_debeGuardarYRetornar() {
        // Arrange
        when(repositorio.existsByNombreIgnoreCase("Laptop Dell XPS"))
            .thenReturn(false);
        when(repositorio.save(any(Producto.class)))
            .thenAnswer(inv -> {
                Producto p = inv.getArgument(0);
                p.setId(1L);
                return p;
            });
    }

```

```
// Act
Producto resultado = servicio.crear(productoValido);

// Assert
assertThat(resultado.getId()).isEqualTo(1L);
assertThat(resultado.isActivo()).isTrue();
verify(repositorio, times(1)).save(any(Producto.class));
}

@Test
@DisplayName("❌ crear: lanza excepción si el nombre ya existe")
void crear_nombreDuplicado_debeLanzarIllegalArgumentException() {
    // Arrange
    when(repositorio.existsByNombreIgnoreCase("Laptop Dell XPS"))
        .thenReturn(true);

    // Act + Assert
    assertThatThrownBy(() -> servicio.crear(productoValido))
        .assertInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("Ya existe un producto");

    // El repositorio NO debe llamar a save()
    verify(repositorio, never()).save(any());
}

// — BUSCAR —————

@Test
@DisplayName("✅ buscarPorId: retorna producto cuando existe")
void buscarPorId_existe_debeRetornarProducto() {
    // Arrange
    Producto esperado = Producto.builder()
        .id(5L)
        .nombre("Mouse Logitech")
        .precio(new BigDecimal("150000.00"))
        .stock(50)
        .activo(true)
        .build();
    when(repositorio.findById(5L)).thenReturn(Optional.of(esperado));

    // Act
    Producto resultado = servicio.buscarPorId(5L);

    // Assert
    assertThat(resultado).isNotNull();
    assertThat(resultado.getNombre()).isEqualTo("Mouse Logitech");
}

@Test
@DisplayName("❌ buscarPorId: lanza ProductoNotFoundException si no existe")
void buscarPorId_noExiste_debeLanzarProductoNotFoundException() {
    // Arrange
    when(repositorio.findById(99L)).thenReturn(Optional.empty());
```

```
// Act + Assert
assertThatThrownBy(() -> servicio.buscarPorId(99L))
    .assertInstanceOf(ProductoNotFoundException.class)
    .hasMessageContaining("99");
}

// — DESACTIVAR —————

@Test
@DisplayName("✅ desactivar: cambia activo=false sin eliminar de BD")
void desactivar_productoExistente_debePersistirCambio() {
    // Arrange
    Producto activo = Producto.builder()
        .id(3L)
        .nombre("Teclado Mecánico")
        .precio(new BigDecimal("320000.00"))
        .stock(20)
        .activo(true)
        .build();
    when(repositorio.findById(3L)).thenReturn(Optional.of(activo));
    when(repositorio.save(any())).thenReturn(activo);

    // Act
    servicio.desactivar(3L);

    // Assert
    assertThat(activo.isActivo()).assertFalse();
    verify(repositorio).save(activo); // se persistió el cambio
}

// — ACTUALIZAR —————

@Test
@DisplayName("❌ actualizar: lanza excepción si el nuevo nombre está en uso")
void actualizar_nombreOcupado_debeLanzarExcepcion() {
    // Arrange
    Producto existente = Producto.builder()
        .id(1L).nombre("Monitor LG").precio(BigDecimal.TEN).stock(5).build();
    Producto nuevosDatos = Producto.builder()
        .nombre("Teclado Cherry").precio(BigDecimal.TEN).stock(5).build();

    when(repositorio.findById(1L)).thenReturn(Optional.of(existente));
    when(repositorio.existsByNombreIgnoreCase("Teclado
Cherry")).thenReturn(true);

    // Act + Assert
    assertThatThrownBy(() -> servicio.actualizar(1L, nuevosDatos))
        .assertInstanceOf(IllegalArgumentException.class)
        .hasMessageContaining("ya está en uso");
}
}
```

11. Pruebas de integración (MockMvc)

Archivo: `src/test/java/com/uniminuto/productos/integration/ProductoControllerIntTest.java`

```
package com.uniminuto.productos.integration;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.uniminuto.productos.model.Producto;
import com.uniminuto.productos.repository.ProductoRepository;
import org.junit.jupiter.api.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.math.BigDecimal;

import static org.hamcrest.Matchers.*;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

/**
 * PRUEBAS DE INTEGRACIÓN – Controlador REST + Servicio + BD H2
 *
 * @SpringBootTest          → levanta el contexto Spring completo
 * @AutoConfigureMockMvc    → configura MockMvc para simular peticiones HTTP
 *
 * ✅ Prueban: la cadena completa Controller → Service → Repository → H2
 * ❌ No prueban: cliente HTTP real (usa MockMvc, no Postman)
 * 🛠 No usan Mockito: todos los componentes son REALES
 *
 * ¿Qué detectan que las unitarias no pueden?
 * • Errores de mapeo JSON ↔ objeto Java
 * • Validaciones de @Valid en el controlador
 * • Comportamiento correcto de los códigos HTTP
 * • Integración real con JPA e H2
 *
 * Ejecución: mvn test -Dtest=ProductoControllerIntTest
 */
@SpringBootTest
@AutoConfigureMockMvc
@DisplayName("ProductoController – Pruebas de Integración")
class ProductoControllerIntTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper; // serializa/deserializa JSON
}
```



```

@Autowired
private ProductoRepository repositorio;

@BeforeEach
void limpiarBD() {
    repositorio.deleteAll(); // BD limpia antes de cada test
}

// — POST /api/productos —————

@Test
@DisplayName("✅ POST: crea producto y retorna 201 Created")
void crearProducto_datosValidos_retorna201() throws Exception {
    Producto nuevo = Producto.builder()
        .nombre("Silla Ergonómica")
        .descripcion("Silla para trabajo remoto")
        .precio(new BigDecimal("850000.00"))
        .stock(5)
        .build();

    mockMvc.perform(post("/api/productos")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(nuevo)))
        .andExpect(status().isCreated()) // 201
        .andExpect(jsonPath("$.id").exists()) // tiene ID asignado
        .andExpect(jsonPath("$.nombre").value("Silla Ergonómica"))
        .andExpect(jsonPath("$.activo").value(true)); // activo por defecto
}

@Test
@DisplayName("❌ POST: retorna 400 si falta el nombre")
void crearProducto_sinNombre_retorna400() throws Exception {
    Producto invalido = Producto.builder()
        // nombre omitido → @NotBlank debe rechazarlo
        .precio(new BigDecimal("100000.00"))
        .stock(1)
        .build();

    mockMvc.perform(post("/api/productos")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(invalido)))
        .andExpect(status().isBadRequest()); // 400
}

// — GET /api/productos —————

@Test
@DisplayName("✅ GET: lista productos activos")
void listarProductos_retornaListaActivos() throws Exception {
    // Arrange: insertamos 2 productos directamente en BD
    repositorio.save(Producto.builder()
        .nombre("Webcam HD").precio(new
BigDecimal("200000")).stock(15).activo(true).build());

```

```

        repositorio.save(Producto.builder()
            .nombre("Auriculares BT").precio(new
BigDecimal("350000")).stock(8).activo(true).build());

// Act + Assert
mockMvc.perform(get("/api/productos"))
    .andExpect(status().isOk()) // 200
    .andExpect(jsonPath("$", hasSize(2)))
    .andExpect(jsonPath("$[0].nombre").value("Webcam HD"));
}

// — GET /api/productos/{id} —————

@Test
@DisplayName("❌ GET /{id}: retorna 404 si el producto no existe")
void buscarPorId_noExiste_retorna404() throws Exception {
    mockMvc.perform(get("/api/productos/9999"))
        .andExpect(status().isNotFound()) // 404
        .andExpect(jsonPath("$.error").exists()); // mensaje de error
}

// — PUT /api/productos/{id} —————

@Test
@DisplayName("✅ PUT: actualiza producto existente y retorna 200")
void actualizarProducto_existente_retorna200() throws Exception {
    // Arrange: guardamos primero
    Producto guardado = repositorio.save(Producto.builder()
        .nombre("Impresora Básica")
        .precio(new BigDecimal("500000"))
        .stock(3)
        .activo(true)
        .build());

    // Datos de actualización
    Producto actualizado = Producto.builder()
        .nombre("Impresora Pro") // nombre cambiado
        .precio(new BigDecimal("750000")) // precio actualizado
        .stock(3)
        .build();

    // Act + Assert
    mockMvc.perform(put("/api/productos/" + guardado.getId())
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(actualizado)))
        .andExpect(status().isOk())
        .andExpect(jsonPath("$.nombre").value("Impresora Pro"))
        .andExpect(jsonPath("$.precio").value(750000.00));
}

// — DELETE /api/productos/{id} —————

@Test
@DisplayName("✅ DELETE: desactiva producto y retorna 204 No Content")

```

```

void eliminarProducto_existente_retorna204() throws Exception {
    // Arrange
    Producto guardado = repositorio.save(Producto.builder()
        .nombre("Cable USB")
        .precio(new BigDecimal("25000"))
        .stock(100)
        .activo(true)
        .build());

    // Act
    mockMvc.perform(delete("/api/productos/" + guardado.getId()))
        .andExpect(status().isNoContent()); // 204

    // Assert: el producto sigue en BD pero inactivo
    Producto verificado =
    repositorio.findById(guardado.getId()).orElseThrow();

    org.assertj.core.api.Assertions.assertThat(verificado.getActivo()).isFalse();
}
}

```

12. Pruebas de aceptación (Cucumber BDD)

12.1 Archivo de escenarios

Archivo: `src/test/resources/features/productos.feature`

```

# language: es

Característica: Gestión de productos del catálogo
    Como administrador del catálogo
    Quiero poder crear, consultar, actualizar y desactivar productos
    Para mantener actualizado el inventario del sistema

    Escenario: Registrar un producto nuevo exitosamente
        Dado que no existe ningún producto con nombre "Audífonos Sony"
        Cuando registro un producto con nombre "Audífonos Sony", precio 280000 y stock
20
        Entonces el sistema confirma la creación con código 201
        Y el producto aparece como activo en el catálogo

    Escenario: Rechazar registro de producto con nombre duplicado
        Dado que ya existe un producto registrado con nombre "Monitor Samsung"
        Cuando intento registrar otro producto con nombre "Monitor Samsung" y precio
1200000
        Entonces el sistema retorna código de error 409
        Y el mensaje de error indica que el nombre ya está en uso

    Escenario: Consultar producto existente por ID
        Dado que existe un producto "Teclado HP" con precio 180000 registrado

```

Cuando consulto el producto por su ID
 Entonces el sistema retorna el producto con nombre "Teclado HP"
 Y el código de respuesta es 200

Escenario: Desactivar un producto del catálogo

Dado que existe un producto activo "Cargador Universal" con precio 45000
 Cuando desactivo el producto "Cargador Universal"
 Entonces el sistema retorna código 204
 Y el producto "Cargador Universal" ya no aparece en el listado de activos

12.2 Step Definitions

Archivo: `src/test/java/com/uniminuto/productos/acceptance/steps/ProductoSteps.java`

```
package com.uniminuto.productos.acceptance.steps;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.uniminuto.productos.model.Producto;
import com.uniminuto.productos.repository.ProductoRepository;
import io.cucumber.java.Before;
import io.cucumber.java.es.*;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.AutoConfigureMockMvc;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;
import org.springframework.test.web.servlet.MvcResult;

import java.math.BigDecimal;

import static org.assertj.core.api.Assertions.assertThat;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.*;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

/**
 * STEP DEFINITIONS para los escenarios Cucumber.
 *
 * Cada método mapea una línea del .feature a código ejecutable.
 * Usa el contexto Spring real (MockMvc + BD H2) para validar
 * comportamiento observable desde perspectiva del usuario/cliente.
 */
@SpringBootTest
@AutoConfigureMockMvc
public class ProductoSteps {

    @Autowired private MockMvc mockMvc;
    @Autowired private ObjectMapper objectMapper;
    @Autowired private ProductoRepository repository;
```

```
// Estado compartido entre pasos del mismo escenario
private MvcResult ultimaRespuesta;
private Long ultimoIdCreado;

@Before
public void limpiarEstado() {
    repositorio.deleteAll();
    ultimaRespuesta = null;
    ultimoIdCreado = null;
}

// — DADOS —

@Dado("que no existe ningún producto con nombre {string}")
public void dadoNoExisteProductoConNombre(String nombre) {
    assertThat(repositorio.existsByNombreIgnoreCase(nombre)).isFalse();
}

@Dado("que ya existe un producto registrado con nombre {string}")
public void dadoExisteProductoConNombre(String nombre) {
    repositorio.save(Producto.builder()
        .nombre(nombre)
        .precio(new BigDecimal("500000"))
        .stock(10)
        .activo(true)
        .build());
}

@Dado("que existe un producto {string} con precio {int} registrado")
public void dadoExisteProductoConPrecio(String nombre, int precio) throws
Exception {
    Producto guardado = repositorio.save(Producto.builder()
        .nombre(nombre)
        .precio(new BigDecimal(precio))
        .stock(5)
        .activo(true)
        .build());
    ultimoIdCreado = guardado.getId();
}

@Dado("que existe un producto activo {string} con precio {int}")
public void dadoExisteProductoActivo(String nombre, int precio) throws
Exception {
    Producto guardado = repositorio.save(Producto.builder()
        .nombre(nombre)
        .precio(new BigDecimal(precio))
        .stock(10)
        .activo(true)
        .build());
    ultimoIdCreado = guardado.getId();
}

// — CUANDOS —
```

```
@Cuando("registro un producto con nombre {string}, precio {int} y stock {int}")
public void cuandoRegistroProducto(String nombre, int precio, int stock)
throws Exception {
    Producto nuevo = Producto.builder()
        .nombre(nombre)
        .precio(new BigDecimal(precio))
        .stock(stock)
        .build();

    ultimaRespuesta = mockMvc.perform(post("/api/productos")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(nuevo)))
        .andReturn();

    // Capturamos el ID si la creación fue exitosa
    if (ultimaRespuesta.getResponse().getStatus() == 201) {
        Producto creado = objectMapper.readValue(
            ultimaRespuesta.getResponse().getContentAsString(),
            Producto.class);
        ultimoIdCreado = creado.getId();
    }
}

@Cuando("intento registrar otro producto con nombre {string} y precio {int}")
public void cuandoIntentoDuplicar(String nombre, int precio) throws Exception
{
    Producto duplicado = Producto.builder()
        .nombre(nombre)
        .precio(new BigDecimal(precio))
        .stock(1)
        .build();

    ultimaRespuesta = mockMvc.perform(post("/api/productos")
        .contentType(MediaType.APPLICATION_JSON)
        .content(objectMapper.writeValueAsString(duplicado)))
        .andReturn();
}

@Cuando("consulta el producto por su ID")
public void cuandoConsultaPorId() throws Exception {
    ultimaRespuesta = mockMvc.perform(get("/api/productos/" + ultimoIdCreado))
        .andReturn();
}

@Cuando("desactivo el producto {string}")
public void cuandoDesactivoProducto(String nombre) throws Exception {
    ultimaRespuesta = mockMvc.perform(delete("/api/productos/" +
ultimoIdCreado))
        .andReturn();
}

// — ENTONCESS —
```

```
@Entonces("el sistema confirma la creación con código {int}")
public void entoncesCodigoRespuesta(int codigoEsperado) {

assertThat(ultimaRespuesta.getResponse().getStatus()).isEqualTo(codigoEsperado);
}

@Y("el producto aparece como activo en el catálogo")
public void yProductoActivoEnCatalogo() throws Exception {
    Producto creado = objectMapper.readValue(
        ultimaRespuesta.getResponse().getContentAsString(), Producto.class);
    assertThat(creado.getActivo()).isTrue();
}

@Entonces("el sistema retorna código de error {int}")
public void entoncesCodigoDeError(int codigo) {
    assertThat(ultimaRespuesta.getResponse().getStatus()).isEqualTo(codigo);
}

@Y("el mensaje de error indica que el nombre ya está en uso")
public void yMensajeDeNombreDuplicado() throws Exception {
    String cuerpo = ultimaRespuesta.getResponse().getContentAsString();
    assertThat(cuerpo).containsIgnoringCase("Ya existe");
}

@Entonces("el sistema retorna el producto con nombre {string}")
public void entoncesRetornaProductoConNombre(String nombreEsperado) throws
Exception {
    Producto producto = objectMapper.readValue(
        ultimaRespuesta.getResponse().getContentAsString(), Producto.class);
    assertThat(producto.getNombre()).isEqualTo(nombreEsperado);
}

@Y("el código de respuesta es {int}")
public void yCodigoDeRespuesta(int codigo) {
    assertThat(ultimaRespuesta.getResponse().getStatus()).isEqualTo(codigo);
}

@Entonces("el sistema retorna código {int}")
public void entoncesCodigoHttp(int codigo) {
    assertThat(ultimaRespuesta.getResponse().getStatus()).isEqualTo(codigo);
}

@Y("el producto {string} ya no aparece en el listado de activos")
public void yProductoNoAparece(String nombre) {
    boolean existeActivo = repositorio.findByActivoTrue()
        .stream()
        .anyMatch(p -> p.getNombre().equals(nombre));
    assertThat(existeActivo).isFalse();
}
}
```

12.3 Suite de Cucumber

Archivo: `src/test/java/com/uniminuto/productos/acceptance/AceptacionSuite.java`

```
package com.uniminuto.productos.acceptance;

import org.junit.platform.suite.api.*;

@Suite
@IncludeEngines("cucumber")
@SelectClasspathResource("features")
@ConfigurationParameter(
    key = "cucumber.glue",
    value = "com.uniminuto.productos.acceptance.steps"
)
@ConfigurationParameter(
    key = "cucumber.plugin",
    value = "pretty, html:target/cucumber-report.html"
)
@ConfigurationParameter(
    key = "cucumber.publish.quiet",
    value = "true"
)
public class AceptacionSuite {
    // Vacía: Cucumber usa las anotaciones para su configuración
}
```

13. Cómo ejecutar todo

Ejecutar todas las pruebas

```
mvn test
```

Ejecutar solo un tipo

```
# Solo unitarias
mvn test -Dtest=ProductoServiceTest

# Solo integración
mvn test -Dtest=ProductoControllerIntTest

# Solo aceptación (Cucumber)
mvn test -Dtest=AceptacionSuite
```

Ejecutar la aplicación (modo desarrollo)


```
mvn spring-boot:run
```

Endpoints disponibles en <http://localhost:8080>:

Método	Ruta	Descripción
POST	/api/productos	Crear producto
GET	/api/productos	Listar activos
GET	/api/productos/{id}	Buscar por ID
GET	/api/productos/buscar?nombre=x	Buscar por nombre
PUT	/api/productos/{id}	Actualizar
DELETE	/api/productos/{id}	Desactivar (soft delete)

Ver reporte Cucumber

Después de correr `mvn test`, el reporte HTML estará en:

```
target/cucumber-report.html
```

14. Tabla comparativa de pruebas

Dimensión	Unitarias	Integración	Aceptación (BDD)
Clase de ejemplo	<code>ProductoServiceTest</code>	<code>ProductoControllerIntTest</code>	<code>productos.feature</code>
¿Qué verifica?	Lógica de negocio aislada	Cadena Controller→Service→BD	Criterios de usuario (HTTP completo)
Spring Context	✗ No se levanta	✓ Completo	✓ Completo
Base de datos	✗ Mock del repositorio	✓ H2 real en memoria	✓ H2 real en memoria
Usa Mockito	✓ Sí (mock del repo)	✗ No (componentes reales)	✗ No (componentes reales)
Velocidad	< 100 ms	500 ms – 2 s	1 – 3 s por escenario
Herramientas	JUnit 5 + Mockito + AssertJ	JUnit 5 + MockMvc + AssertJ	Cucumber + MockMvc
Lenguaje	Java (código)	Java (código)	Gherkin (lenguaje natural)

Dimensión	Unitarias	Integración	Aceptación (BDD)
¿Quién los lee?	Solo desarrolladores	Desarrolladores / QA	Desarrolladores + PO + Cliente
Detecta errores en	Reglas de negocio, guard clauses	Serialización JSON, validaciones HTTP, SQL	Flujos completos, criterios de aceptación
No detecta	Errores de integración, SQL	Bugs de lógica pura aislada	Dónde exactamente falló
Proporción recomendada	70 %	20 %	10 %
Relación ISO 25010	Funcionalidad, Mantenibilidad	Fiabilidad, Compatibilidad	Adecuación funcional, Usabilidad

Regla clave: una prueba de integración que usa Mockito para el repositorio es, en realidad, una prueba unitaria disfrazada. Si no hay BD real involucrada, no valida la integración.

Documento generado para el curso Estándares y Métricas de Calidad — UNIMINUTO